

Cloud DevOpsSec

Program Analysis

Outline

- Program Analysis
 - What is static and dynamic analysis?
 - The importance of static analysis
- Web Application Security Risks

DevOps SDLC



source: adapted from <https://levelup.gitconnected.com/top-15-devsecops-tools-for-an-enterprise-ci-cd-pipeline-bd865b47ed5f>

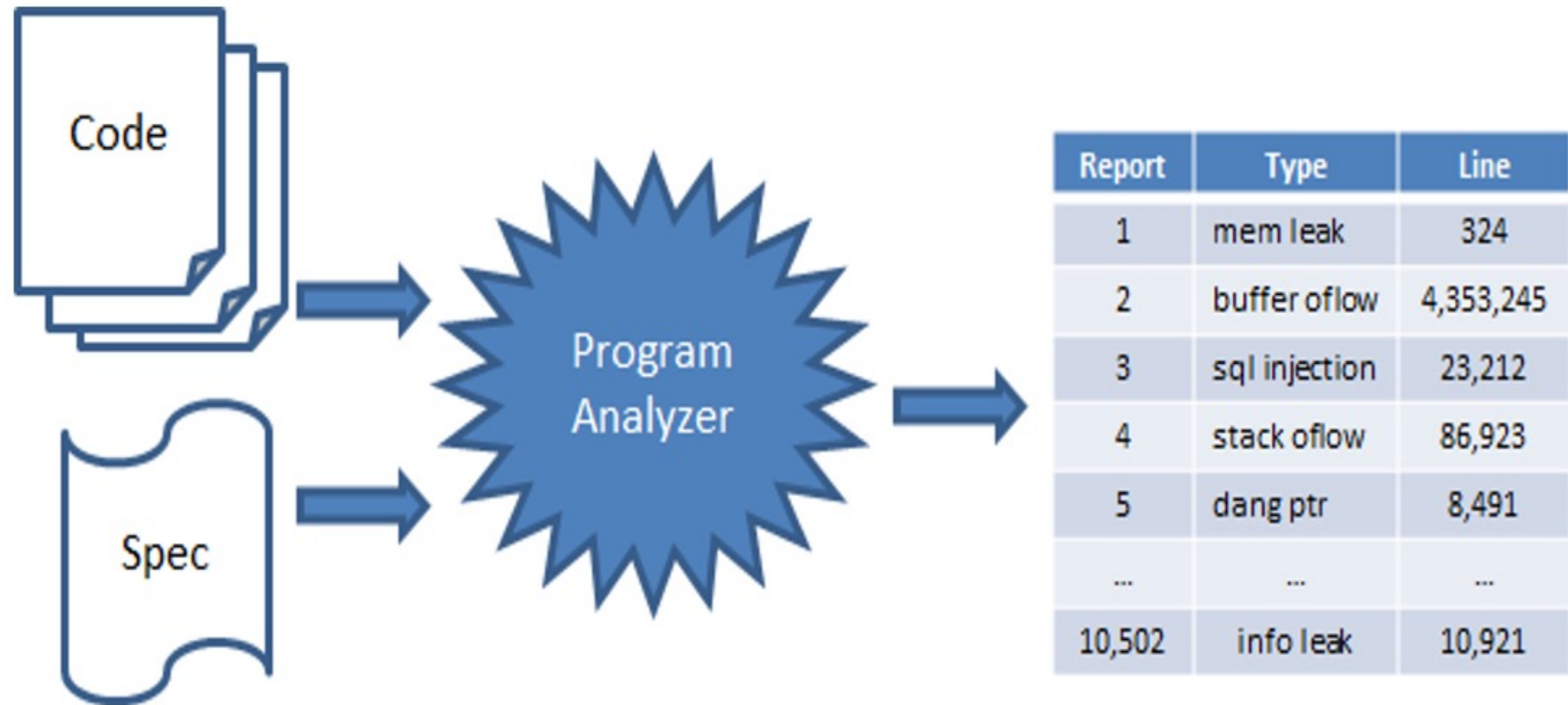
DevOps SDLC

Static Analysis



source: adapted from <https://levelup.gitconnected.com/top-15-devsecops-tools-for-an-enterprise-ci-cd-pipeline-bd865b47ed5f>

Program Analysis



Program Analysis

- Dynamic Program Analysis
- Static Program Analysis

Program Analysis

- Dynamic Program Analysis
 - Dynamic Application Security Testing (DAST)
- Static Program Analysis
 - Static Application Security Testing (SAST)

Dynamic Program Analysis

- Is a popular computer program analysis technique in which the reasoning about a program's properties/behavior is made at runtime, during the program's execution

Dynamic Program Analysis – Applications

- Bugs detection
 - e.g., memory leaks, deadlocks, detect undefined value, etc.
- Profiling
 - e.g., call graph execution, memory allocations, etc.
- Invariant detection
- Performance analysis
- Security analysis
 - e.g., Intrusion detection and prevention

Dynamic application security testing (DAST)

- Is performed at runtime and attempts to simulate attacks and analyse the response of the application to these attacks
 - Subsequently, instrumented code is executed, various inputs are chosen, and the response of the application is analysed

Dynamic application security testing (DAST)

- Google Sanitizers: AddressSanitizer, MemorySanitizer, ThreadSanitizer, LeakSanitizer, etc.
 - A memory error detector for C/C++ able to identify various memory issues
 - e.g., buffer overflows, memory leaks, use after free, etc.

Static Program Analysis

- Is a popular computer program analysis technique in which the reasoning about a program's properties/behavior is made without running the program
- Static analysis is usually carried out at the implementation level in the SDLC

Static Program Analyzer

- A program that reasons about the behavior of other programs
- Checks all the possible executions of a program and provides guarantees about the program properties/behavior

Static Program Analysis – Applications

- Optimizing compilers for generating efficient code
- Automatic error detection
 - e.g., uninitialized data, memory leaks, concurrency issues
- Automatic detection of common security vulnerabilities
- Compliance with safety-related coding standards
- Compliance with coding guidelines and naming conventions, and maintainability requirements
 - e.g., compliance with the coding rules established in the organisation

Static Program Analysis – Applications

- Optimizing compilers for generating efficient code – automatic optimization
 - Is there any dead code?
 - Eliminate dead code – reduce code size
 - Is the value of an expression the same in every iteration of a loop?
 - Hoisting – move the loop-invariant code outside of the loop
 - Does the value of a variable depend on the input?
 - If not, precompute the value

Static Program Analysis – Applications

- Automatic error detection
 - Are all variables initialized before they are read?
 - Does there exist an input that leads to a null pointer dereference, division-by-zero, or arithmetic overflow?
 - Are arrays always accessed within their bounds?
 - How large can the heap become during the program execution?
 - Correctness related question example: Does the program throw a SQLiteException for some input?

Static Program Analysis – Applications

- Automatic error detection
 - Information flow correctness
 - Can an input value from an untrusted user flow unchecked to file system operations?
 - Violates integrity
 - Can secret/sensitive information become publicly available?
 - Violates confidentiality
 - Are data races possible?
 - Are deadlocks possible?

Static Program Analysis – Applications

- Analysis for program development
 - Support debugging, refactoring, program understanding
 - Where can a method be invoked from?
 - What types of values can a variable have?

Static Code Analyzers – Limitations

- False positives
 - May detect errors that do not exist
- False negatives
 - May fail to detect real errors that exist
- Lack of developer intent
 - e.g.,

```
def divide(self):  
    return self.number1 + self.number2
```

Static Application Security Testing (SAST)

- It examines the application code, byte code or binaries in order to check for vulnerabilities.
- SAST is focused on the analysis of the application data, the identification of tainted data and data flows in the application and identification of invalid flows.
- Static analysis is not really testing the functionality (i.e. no code running)
- e.g., tainted flow analysis

SAST – Tainted Flow Analysis (TFA)

- The root cause of many attacks is represented by the trust in invalidated input
- TFA – tracks how values might flow between different locations, and can detect illegal flows
 - *Illegal flow* – flow where tainted data is used where untainted data is expected
 - Potential security vulnerability

SAST – Tainted Flow Analysis (TFA)

- Legal flow example

```
void function1(tainted int);
```

```
untainted int value1 = ...;
```

```
function1 (value1);
```

SAST – Tainted Flow Analysis (TFA)

- Illegal flow example

```
void function2(untainted int);
```

```
tainted int value2 = ...;
```

```
function2 (value2);
```

Static Analyzers

- Select an analyser that focuses on finding vulnerabilities in the code
- Various tools
 - Open-source or proprietary
 - Choose a tool that has a lower number of false positive results

Static Analyzers

- lint/linter – a static code analysis tool used to detect programming errors, bugs, stylistic errors, etc.
 - e.g., Pychecker and Pylint – static analysis tools for Python programs
- Zoncolan – Facebook
 - Static Application Security Testing (SAST) Tool
 - Analyze source code to identify security flaws

Security Analysis

- In practice, static and dynamic analysis are often combined in security testing as they complement each other.
- e.g., static analysis can be used in the selection of the tested inputs in a dynamic analysis
 - Code review can also be used for this purpose.

Security Static Analyzers

- Source Code Analysis Tools compiled by OWASP (Open Worldwide Application Security Project)
 - https://owasp.org/www-community/Source_Code_Analysis_Tools
- Source Code Security Analyzers compiled by NIST (National Institute of Standards and Technology)
 - https://samate.nist.gov/index.php/Source_Code_Security_Analyzers.html

OWASP: Top 10 Web Application Security Risks (2021)

- *Broken Access Control*
- *Cryptographic Control*
- *Injection*
- *Insecure Design*
- *Security Misconfiguration*
- *Vulnerable and Outdated Components*
- *Identification and Authentication Failure*
- *Software and Data Integrity Issues*
- *Security Logging & Monitoring Failures*
- *Server-Side Request Forgery (SSRF)*

OWASP: Top 10 Web Application Security Risks (2017)

- *Injection*
- *Broken Authentication*
- *Sensitive Data Exposure*
- *XML External Entities (XXE)*
- *Broken Access Control*
- *Security Misconfiguration*
- *Cross-Site Scripting (XSS)*
- *Insecure Deserialization*
- *Using Components with Known Vulnerabilities*
- *Insufficient Logging & Monitoring*

OWASP: Top 10 Web Application Security Risks

Injection

- “Injection flaws, such as SQL, NoSQL, OS, and LDAP injection, occur when untrusted data is sent to an interpreter as part of a command or query
- The attacker’s hostile data can trick the interpreter into executing unintended commands or accessing data without proper authorization”

OWASP: Top 10 Web Application Security Risks

Injection

- Impacts
 - Injection can result in data loss, corruption, or disclosure of (sensitive) data, loss of accountability, or denial of access
 - Injection can sometimes lead to complete host takeover.

OWASP: Top 10 Web Application Security Risks

Injection

- An application is vulnerable to injection attacks if
 - Data provided by end-user is not validated, filtered, or sanitized by the application
 - Dynamic queries or non-parameterized calls without context-aware escaping are used directly
 - Hostile data is used within object-relational mapping (ORM) search parameters to extract sensitive records
 - Hostile data is directly used or concatenated, such that the SQL or command contains both structure and hostile data in dynamic queries, commands, or stored procedures

OWASP: Top 10 Web Application Security Risks

Injection

- Preventing injections
 - Keep data separate from commands and queries
 - Use a safe API, which avoids the use of the interpreter entirely or provides a parameterized interface
 - Migrate to use Object Relational Mapping Tools (ORMs)
 - Use positive or “whitelist” server-side input validation
 - For any residual dynamic queries, escape special characters using the specific escape syntax for that interpreter
 - Use LIMIT and other SQL controls within queries to prevent mass disclosure of records in case of a SQL injection

OWASP: Top 10 Web Application Security Risks

Injection

- Detecting injections
 - Source code review
 - Static application security testing
 - Dynamic application security testing

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- Previously known as *Broken Authentication*
- “Application functions related to authentication and session management are often implemented incorrectly, allowing attackers to compromise passwords, keys, or session tokens, or to exploit other implementation flaws to assume other users’ identities temporarily or permanently”

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- Previously known as *Broken Authentication*
- Now also includes Common Weakness Enumerations (CWEs) related to identification failures:
 - E.g., *CWE-297: Improper Validation of Certificate with Host Mismatch*; *CWE-287: Improper Authentication*; and *CWE-384: Session Fixation*.

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- Impacts

- Attackers have to gain access to only a small number of accounts or an admin account to compromise the entire system
- e.g., money laundering, social security fraud, identity theft, or disclosure of sensitive data

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- An application is vulnerable to authentication attacks if
 - Permits automated attacks such as *credential stuffing*, where the attacker has a list of valid usernames and passwords
 - Permits brute force or other automated attacks
 - Permits default, weak, or well-known passwords
 - e.g., “password”, “passwd”, “Password1”, “admin/admin”
 - Uses weak or ineffective credential recovery and forgot-password processes
 - e.g., “knowledge-based answers” – is not safe

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- An application is vulnerable to authentication attacks if
 - Uses plain text, encrypted, or weakly hashed passwords
 - Has missing or ineffective multi-factor authentication
 - Session ID are presented in the URL
 - Does not rotate Session IDs (reuse session IDs) after successful login
 - User sessions or authentication tokens are not properly invalidated during logout or a period of inactivity

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- Preventing broken authentication vulnerabilities
 - Implement multi-factor authentication
 - prevents automated credential stuffing, brute force, and reuse of stolen credentials
 - Do not deploy your application with any default credentials
 - Implement weak-password checks
 - e.g., test new or changed passwords
 - Align password length, complexity and rotation policies with NIST 800-63b's guidelines

OWASP: Top 10 Web Application Security Risks

Identification and Authentication Failures

- Preventing broken authentication vulnerabilities
 - Ensure registration, credential recovery, and API pathways are hardened against account enumeration attacks by *using the same messages for all outcomes*
 - Limit or increasingly delay failed login attempts. Log all failures and alert administrators when credential stuffing, brute force, or other attacks are detected.
 - Use a server-side, secure, built-in session manager that generates a new random session ID with high entropy after login
 - Session IDs should not be in the URL, be securely stored and invalidated after logout, idle, and absolute timeouts

OWASP: Top 10 Web Application Security Risks

Sensitive Data Exposure

- Many web applications and APIs do not properly protect sensitive data
 - e.g., financial data, health records, personally identifiable information
- Attackers may steal or modify weakly protected data to conduct different crimes
 - e.g., credit card fraud, identity theft, etc.
- Sensitive data requires special handling when exchanged with the browser
 - e.g., encryption at rest or in transit

OWASP: Top 10 Web Application Security Risks

Sensitive Data Exposure

- Impacts
 - Compromises all data that should have been protected
 - e.g., personally identifiable information

OWASP: Top 10 Web Application Security Risks

Sensitive Data Exposure

- An application is vulnerable to attacks if
 - Data is transmitted in clear text
 - e.g., HTTP, SMTP, FTP
 - Usage of old or weak cryptographic algorithms are used either directly or via old code
 - Encryption not enforced?
 - The user agent (e.g., app, mail client) does not verify the validity of the received server certificate

OWASP: Top 10 Web Application Security Risks

Sensitive Data Exposure

- Preventing sensitive data exposure vulnerabilities
 - Classify data processed, stored or transmitted by an application
 - Identify which data is sensitive according to privacy laws (e.g., EU's GDPR), regulatory requirements (Payment Card Industry Data Security Standard (PCI DSS)) or business needs
 - Apply controls according to the classification
 - Don't store any sensitive data that is not necessary

OWASP: Top 10 Web Application Security Risks

Sensitive Data Exposure

- Preventing sensitive data exposure vulnerabilities
 - Encrypt all sensitive data at rest
 - Encrypt all data in transit with secure protocols
 - e.g., Transport Layer Security (TLS), HTTP Strict Transport Security (HSTS)
 - Disable caching for responses that contain sensitive data
 - Store passwords using strong hashing functions
 - Passwords should be hashed, NOT encrypted!

OWASP: Top 10 Web Application Security Risks

XML External Entities (XXE)

- Attackers can exploit vulnerable XML processors if they can upload XML or include content in an XML document
 - e.g., exploit vulnerable code, dependencies, etc.
- “External entities can be used to disclose internal files using the file URI handler, internal file shares, internal port scanning, remote code execution, and denial of service attacks”

OWASP: Top 10 Web Application Security Risks

XML External Entities (XXE)

- Impacts
 - Can be used to
 - disclose internal files using the file URI handler, internal file shares, internal port scanning
 - Execute Remote code
 - Perform denial-of-service attacks

OWASP: Top 10 Web Application Security Risks

XML External Entities (XXE)

- An application is vulnerable to attacks if
 - The application accepts XML directly or XML uploads from untrusted sources
 - The application uses Security Assertion Markup Language (SAML) – an open standard that enables single sign-on (SSO)
 - SAML uses XML for identity assertions, and may be vulnerable
- The application uses SOAP version < 1.2

OWASP: Top 10 Web Application Security Risks

XML External Entities (XXE)

- Preventing XXE vulnerabilities
 - Use less complex data formats (e.g. JSON) and avoid serialization of sensitive data
 - Patch or upgrade all XML processors and libraries used by the application or on the underlying OS
 - Implement positive (“whitelisting”) server-side input validation, filtering, or sanitization to prevent hostile data within XML documents
 - Validate all XML file uploads

OWASP: Top 10 Web Application Security Risks

Broken Access Control

- Restrictions on what authenticated users are allowed to do are not enforced
 - Attackers can exploit these flaws to access unauthorized functionality and/or data
 - e.g., access other users' accounts, view sensitive files, modify other users' data, change access rights
- Due to lack of automated detection and lack of effective functional testing
 - Testing is the best approach to detect missing or ineffective access control

OWASP: Top 10 Web Application Security Risks

Broken Access Control

- Impacts
 - Users use privileged functions, or create, access, update, or delete every record
 - Attackers act as users or administrators

OWASP: Top 10 Web Application Security Risks

Broken Access Control

- An application is vulnerable to attacks if
 - One can bypass access control checks by modifying the URL, internal application state, or the HTML page
 - The application permits viewing or editing another user's account
 - Privilege escalation/ Privilege elevation
 - e.g., act as a registered user without being logged in
 - e.g., act as an admin when logged in as a standard user

OWASP: Top 10 Web Application Security Risks

Broken Access Control

- An application is vulnerable to attacks if
 - Cross-Origin Resource Sharing (CORS) allows unauthorized API access
 - Force browsing to authenticated pages as an unauthenticated user
 - Force browsing to privileged pages as a standard user

OWASP: Top 10 Web Application Security Risks

Broken Access Control

- Preventing broken access control vulnerabilities
 - With the exception of public resources, deny by default
 - Implement access control mechanisms once and re-use them throughout the application
 - Model access controls should enforce record ownership, instead of assuming that the user can perform CRUD operations on any record
 - Log access control failures and alert admins
 - e.g., repeated failures

OWASP: Top 10 Web Application Security Risks

Security Misconfiguration

- “Security misconfiguration is the most commonly seen issue. This is commonly a result of insecure default configurations, incomplete or ad hoc configurations, open cloud storage, misconfigured HTTP headers, and verbose error messages containing sensitive information.
- Not only must all operating systems, frameworks, libraries, and applications be securely configured, but they must be patched/upgraded in a timely fashion.”

OWASP: Top 10 Web Application Security Risks

- *Cross-Site Scripting (XSS)*

- “XSS flaws occur whenever an application includes untrusted data in a new web page without proper validation or escaping, or updates an existing web page with user-supplied data using a browser API that can create HTML or JavaScript
- XSS allows attackers to execute scripts in the victim’s browser which can hijack user sessions, deface web sites, or redirect the user to malicious sites”

OWASP: Top 10 Web Application Security Risks

- *Insecure Deserialization*

- “Insecure deserialization often leads to remote code execution.
- Even if deserialization flaws do not result in remote code execution, they can be used to perform attacks, including replay attacks, injection attacks, and privilege escalation attacks”

OWASP: Top 10 Web Application Security Risks

- *Using Components with Known Vulnerabilities*
 - “Components, such as libraries, frameworks, and other software modules, run with the same privileges as the application.
 - If a vulnerable component is exploited, such an attack can facilitate serious data loss or server takeover.
 - Applications and APIs using components with known vulnerabilities may undermine application defenses and enable various attacks and impacts”

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- Previously known as *Insufficient Logging & Monitoring*
- “Insufficient logging and monitoring, coupled with missing or ineffective integration with incident response, allows attackers to further attack systems, maintain persistence, pivot to more systems, and tamper, extract, or destroy data.
- Most breach studies show time to detect a breach is over 200 days, typically detected by external parties rather than internal processes or monitoring.”

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- Help detect, escalate, and respond to active breaches. Without logging and monitoring, breaches cannot be detected
- A system is vulnerable to information leakage by making logging and alerting events visible to a user or an attacker

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- An application is vulnerable to attacks if there is insufficient logging, detection, monitoring, and active response (1/2):
 - Auditable events, such as logins, failed logins, and high-value transactions, are not logged
 - Warnings and errors generate no, inadequate, or unclear log messages
 - Logs of applications and APIs are not monitored for suspicious activity
 - Logs are only stored locally

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- An application is vulnerable to attacks if there is Insufficient logging, detection, monitoring, and active response (2/2):
 - Appropriate alerting thresholds and response escalation processes are not in place or effective.
 - Penetration testing and scans by dynamic application security testing (DAST) tools (such as OWASP ZAP) do not trigger alerts.
 - The application cannot detect, escalate, or alert for active attacks in real-time or near real-time.

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- Preventing Security Logging and Monitoring Failures (1/2):
 - Ensuring all login, access control, and server-side input validation failures can be logged with sufficient user context to identify suspicious or malicious accounts and held for enough time to allow delayed forensic analysis.
 - Ensuring that logs are generated in a format that log management solutions can easily consume.
 - Ensuring log data is encoded correctly to prevent injections or attacks on the logging or monitoring systems.

OWASP: Top 10 Web Application Security Risks

Security Logging and Monitoring Failures

- Preventing Security Logging and Monitoring Failures (2/2):
 - Ensuring high-value transactions have an audit trail with integrity controls to prevent tampering or deletion, such as append-only database tables or similar.
 - DevSecOps teams should establish effective monitoring and alerting such that suspicious activities are detected and responded to quickly.
 - Establishing or adopting an incident response and recovery plan, such as National Institute of Standards and Technology (NIST) 800-61r2 or later

Resources

- Anders Møller and Michael I. Schwartzbach, 2020, *Static Program Analysis*. Department of Computer Science, Aarhus University. Lecture notes: <https://cs.au.dk/~amoeller/spa/>
- Bishop, M., 2019. *Computer Security: Art and Science*. 2nd Ed., Addison-Wesley. ISBN-10: 0321712331
- PyChecker: Finding Bugs in Other People's Programs: <https://legacy.python.org/workshops/2002-02/papers/02/index.htm> <http://pychecker.sourceforge.net/>
- Pylint: <https://www.pylint.org/>
- Open Worldwide Application Security Project® (OWASP) <https://owasp.org/>
- PEP 8 – Style Guide for Python Code <https://www.python.org/dev/peps/pep-0008/>
- Google Sanitizer <https://github.com/google/sanitizers>